

Training Neural Networks

Applied Deep Learning — Chapter 5

Yin Yang

Foundation Books Project

Roadmap

The Training Loop

Loss Functions

Gradient Descent

Backpropagation

Mini-Batches and SGD

Optimizers

Learning-Rate Schedules

Initialization

Regularization and Weight Decay

Batch Normalization

Stochastic Regularization

Putting It Together on CIFAR-10

Debugging a Training Run

Summary

The Training Loop

Why Training Is the Engineering Loop

Training is where a model earns trust or exposes a bug.

- Compute loss, backpropagate, update parameters, validate.
- Engineering records matter: split, seed, optimizer, schedule, validation accuracy, elapsed time, final-test policy.
- A final accuracy number alone rarely says whether the issue was data, optimization, regularization, or evaluation.

Running example: CIFAR-10 ($32 \times 32 \times 3$, ten classes).

Anatomy of a Training Step (PyTorch)

`model.train()` Set training behavior (matters for BN/dropout).

`logits = model(x)` Forward pass; build computation graph.

`loss = loss_fn(logits, y)` Convert prediction + label to scalar.

`optimizer.zero_grad()` Clear stale gradient buffers.

`loss.backward()` Backpropagate; fill gradient buffers.

`optimizer.step()` Apply update rule using gradients + state.

`model.eval(), torch.no_grad()` Validate without training-mode randomness.

Definition

A *training run* is one execution of a complete recipe: data split, preprocessing, model, initialization, optimizer, schedule, regularization, epoch budget, seed, and hardware context.

Classroom CIFAR-10 split: 45,000 train, 5,000 validation, 10,000 official test. Two runs identical except for the validation split can already differ in reported accuracy.

Loss Functions

Logits, Softmax, Cross-Entropy

Model $f_\theta : x \mapsto z \in \mathbb{R}^{10}$ produces **logits**.

$$p_k = \text{softmax}(z)_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}.$$

Per-example cross-entropy:

$$\ell(\theta; x_i, y_i) = -\log p_{y_i}.$$

Empirical risk:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(\theta; x_i, y_i).$$

Why Cross-Entropy, Not Accuracy

Accuracy changes only when the top class changes.

Cross-entropy changes whenever p_{y_i} does:

- Cat label, $p_{\text{cat}} = 0.540$: $\ell \approx 0.616$.
- Cat label, $p_{\text{cat}} = 0.05$: $\ell \approx 2.996$.

Useful gradient even when accuracy is constant.

The Softmax + Cross-Entropy Gradient

For a one-hot target q and logits z :

$$\frac{\partial \ell}{\partial z_k} = p_k - q_k.$$

- Correct class: gradient negative until $p_k \rightarrow 1$.
- Other classes: gradient positive, pushes their logits down.

Clean gradient is the reason softmax + cross-entropy is the classification standard.

Gradient Descent

For $\theta = (\theta_1, \dots, \theta_m)$:

$$\nabla_{\theta} \mathcal{L}(\theta) = \left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \dots, \frac{\partial \mathcal{L}}{\partial \theta_m} \right).$$

Full-batch gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t), \quad \eta > 0.$$

Negative gradient = direction of steepest local decrease.

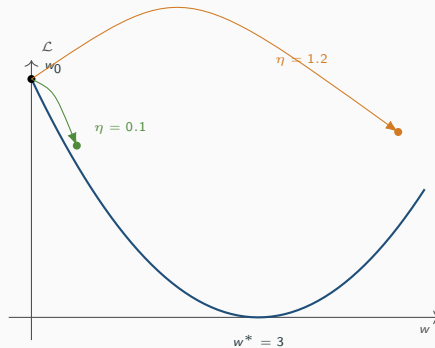
Step Size Matters: $\mathcal{L}(w) = (w - 3)^2$

$w_0 = 0$, derivative -6 .

$\eta = 0.1$: $w_1 = 0 - 0.1(-6) = 0.6$, $\mathcal{L} = 5.76 < 9$.

$\eta = 1.2$: $w_1 = 0 - 1.2(-6) = 7.2$, $\mathcal{L} = 17.64 > 9$.

A locally correct gradient can still fail when the step is too large.



Backpropagation

Computation Graph + Chain Rule

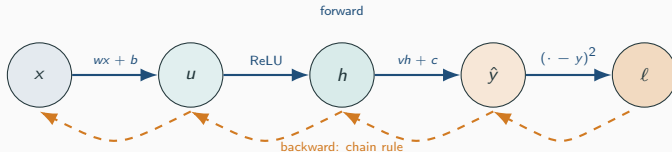
Backprop = efficient chain-rule application over a graph.

- Forward: each node stores its value.
- Backward: each node uses a local derivative rule.
- Frameworks automate this; the meaning of the gradients is what matters.

Common breakage: shape mismatch, accidental detach, wrong train/eval mode, numerical instability ($\rightarrow$ vanishing/exploding gradients).

Worked Backward Pass

$$u = wx + b, \quad h = \text{ReLU}(u), \quad \hat{y} = vh + c, \quad \ell = (\hat{y} - y)^2.$$



$$x=2, w=0.5, b=0.1, v=1.5, c=-0.2, y=1 \Rightarrow u=1.1, h=1.1, \hat{y}=1.45, \ell=0.2025.$$

$$\partial \ell / \partial \hat{y} = 0.9 \Rightarrow \partial \ell / \partial v = 0.99, \partial \ell / \partial c = 0.9; \partial \ell / \partial u = 1.35 \Rightarrow \partial \ell / \partial w = 2.70, \partial \ell / \partial b = 1.35.$$

Mini-Batches and SGD

Full-batch gradient descent is rarely practical.

SGD

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}_{B_t}(\theta_t).$$

B_t is the mini-batch at step t . Iteration = one update. Epoch = one pass through training data.

CIFAR-10 with 45,000 images:

- Batch 64: $\lceil 45000/64 \rceil = 704$ steps/epoch.
- Batch 128: 352 steps/epoch. Batch 256: 176.

Mini-Batch Noise Is Not Pure Noise

Gradients $-1.0, -0.5, 0.2, 1.3$. Full average: 0.

- First two: average $-0.75 \Rightarrow w$ moves up.
- Last two: average $+0.75 \Rightarrow w$ moves down.

Batch losses do not decrease monotonically. Read the smoothed trend.

Batch size changes training. Larger batches: fewer updates per epoch, often need warmup or LR re-tuning.

Optimizers

From SGD to Momentum

Plain SGD: $\theta_{t+1} = \theta_t - \eta g_t$.

SGD with momentum:

$$v_{t+1} = \mu v_t + g_t, \quad \theta_{t+1} = \theta_t - \eta v_{t+1}.$$

$\mu \in [0, 1)$ remembers a decaying gradient history.

Hand example: $\theta_0 = 1.0$, $\eta = 0.1$, $\mu = 0.9$, $g_1 = 0.4$, $g_2 = 0.2$:

$$v_1 = 0.4, \theta_1 = 0.96; \quad v_2 = 0.56, \theta_2 = 0.904.$$

Adam: First and Second Moments

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2.$$

Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{s}_t = \frac{s_t}{1 - \beta_2^t}.$$

Update (elementwise):

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \epsilon}.$$

Adaptivity helps prototypes; LR + validation still need tuning.

AdamW: Decoupled Weight Decay

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \varepsilon}.$$

- Decay step is **separate** from the adaptive gradient step.
- AdamW λ is *not* interchangeable with SGD-with-L2 settings.
- Often a strong default for CIFAR-10 prototypes.

SGD-with-momentum is a strong baseline once the schedule is tuned.

Learning-Rate Schedules

Common Schedules

Take big steps early, smaller steps later.

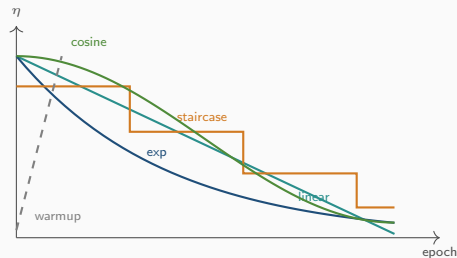
Exponential $\eta_0 d^t$.

Staircase $\eta_0 d^{\lfloor t/\tau \rfloor}$.

Linear $\eta_0(1 - t/T)$.

Cosine $\eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})(1 + \cos \pi t/T)$.

Warmup ramp from small to target.



When Each Schedule Helps

- **Exponential:** conservative steady multiplicative reduction.
- **Staircase:** explicit phases with easy-to-report drop epochs.
- **Linear:** simple for short fixed budgets.
- **Cosine:** smooth long tail without manual drop points.
- **Warmup:** large batches or sensitive normalization dynamics.

Record schedule values with the run. Schedule + optimizer changing together makes the comparison uninterpretable.

Initialization

Why Random, and Why Scale Matters

All-equal weights $\Rightarrow$ identical gradients $\Rightarrow$ redundant units.

Scale problem:

- Too small: signals and gradients shrink.
- Too large: activations and gradients explode.

Initialization sets the starting band.

Xavier / Glorot (symmetric activations)

$$w \sim \mathcal{N}\left(0, \frac{2}{\text{fan-in} + \text{fan-out}}\right), \quad \rho = \sqrt{6/(\text{fan-in} + \text{fan-out})}.$$

He / Kaiming (ReLU family)

$$w \sim \mathcal{N}\left(0, \frac{2}{\text{fan-in}}\right).$$

3×3 conv, 64 input ch, ReLU: $\sigma = \sqrt{2/576} \approx 0.059$.

Residual-Branch Zero- γ Init

In a ResNet block $x + F(x)$ with BN at the end of F :

- Initialize the final BN scale $\gamma=0$ in the learned branch.
- At step 0 the block is approximately the identity.
- Then learning gradually adds corrections.

Not a replacement for Xavier/He on conv weights; complements them in deep residual networks.

Regularization and Weight Decay

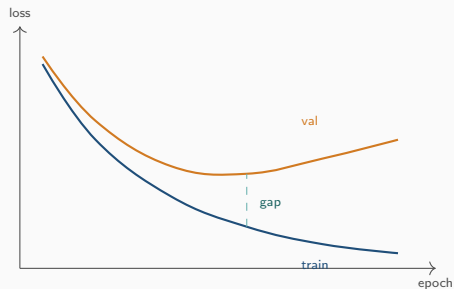
Diagnose the Generalization Gap

Generalization gap = training metric – validation metric.

- Widening gap $\Rightarrow$ evidence to try regularization.
- Don't add regularization just because the option exists.

L2 penalty:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \frac{1}{2} \|\theta\|_2^2.$$



Decoupled Weight Decay

$w = 0.50$, $g = 0.20$, $\eta = 0.10$, $\lambda = 0.01$:

$$w_{\text{new}} = (1 - \eta\lambda)w - \eta g = 0.999 \cdot 0.50 - 0.020 = 0.4795.$$

- Shrinkage and gradient step are separate operations.
- Some recipes exclude biases and BN parameters from decay — record it.
- Tune λ ; too little $\rightarrow$ wide gap, too much $\rightarrow$ underfit.

Batch Normalization

Batch Normalization

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta.$$

- Train mode: use batch statistics, update running estimates.
- Eval mode: use stored running mean/variance.
- For convolutions, statistics are aggregated per channel across batch and spatial positions.

Forgetting eval mode makes validation depend on batch composition.

Activations (2, 4, 6):

$$\mu_B = 4, \sigma_B^2 = 8/3, \sigma_B \approx 1.633.$$

Normalized: $(-1.225, 0, 1.225)$. With $\gamma = 2, \beta = 0.5$:

$$y = (-1.949, 0.5, 2.949).$$

Small batches make these statistics noisy; report batch size with BN.

Stochastic Regularization

Dropout, DropConnect, Stochastic Depth

Dropout Random mask on *activations*. Inverted dropout scales kept activations by $1/q$ during training.

DropConnect Random mask on *weights*.

Stochastic depth Randomly skip residual blocks; scale survivor by $1/q$ to match expectations.

All change behavior between training and evaluation.

Inverted Dropout Example

$h = (2, 4, 6, 8)$, keep prob $q = 0.8$, mask $m = (1, 0, 1, 1)$:

$$\frac{m \odot h}{q} = (2.5, 0, 7.5, 10).$$

- Expected training activation matches unmasked activation.
- Place dropout **after** BN + activation by default.
- Treat dropout rate as an ablation. BN already has some regularizing effect.

Putting It Together on CIFAR-10

A Recipe Is a Coherent Whole

Before comparing results, fix:

- Split, architecture, initialization.
- Normalization, optimizer, LR schedule.
- Weight decay, stochastic regularization.

Augmentation belongs only in the training pipeline. Validation/test preprocessing remains deterministic.

One Change at a Time

- Make the simplest complete recipe work first.
- Improve optimizer + schedule with the architecture fixed.
- Tune regularization next, holding optimizer + schedule fixed.
- Enlarge the architecture only after the small recipe is understood.

Disappointing val. accuracy usually traces to LR, an accidental training transform in val., or a BN mode error — not model size.

Bounded CIFAR-10 Ablations (illustrative)

Run (ResNet18 stem, 4,096 train, 3 epochs)	Best val. acc.	Time
SGD, lr=0.05, no momentum, wd= $5 \cdot 10^{-4}$, const lr	0.3174	10.06s
+ momentum 0.9	0.3604	10.23s
AdamW, lr= 10^{-3} , wd= $5 \cdot 10^{-4}$, const lr	0.3994	10.66s
+ warmup 1 ep, cosine decay	0.4141	—
+ no weight decay	0.4229	—
- batch normalization	0.2617	—
+ head dropout 0.2	0.4053	—

Table is the *format*, not a target. One change at a time.

Debugging a Training Run

Cheap Failures First

- Can the model overfit a tiny fixed subset (e.g. 100 images)?
- Are class labels and class names correct after batching?
- Does training loss decrease in the first few epochs?
- Is the model in train mode for training and eval mode for validation?
- Are random training transforms excluded from validation/test?
- Is weight decay applied to intended weights only (skip biases/BN)?
- Are batch size, optimizer, schedule, seed, and hardware recorded?
- Is the test set used *only once* at the end?

Match the Failure Mode to the Fix

Underfitting Capacity, training time, regularization strength, LR.

Overfitting Augmentation, weight decay, dropout, early stopping, smaller model.

Unstable optimization Lower LR, add warmup, gradient clipping, better init, normalization.

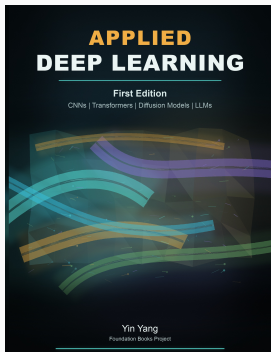
Slow training Batch size, hardware checks — but only when validation evidence remains trustworthy.

Summary

Summary

- Loss defines what training reduces; cross-entropy + softmax are the classification standard.
- Backprop computes gradients efficiently via the chain rule.
- Optimizers (SGD, momentum, Adam, AdamW) and schedules turn gradients into updates.
- Initialization, BN, weight decay, and stochastic regularization improve stability or generalization *when validation evidence supports them*.
- A credible CIFAR-10 result records split, recipe, curves, time, hardware, and a single final-test evaluation.
- One change at a time keeps the evidence trail interpretable.

Next: CNN architectures — stems, blocks, stages, and heads compared on a fixed CIFAR-10 protocol.



Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/applied-deep-learning
Code	github.com/foundation-books/applied-deep-learning-code
Print	amazon.com/dp/B0GZF7QRSH
Kindle	amazon.com/dp/B0GZF9221X
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.